CS23336/GE23231/GE23233-IPP/PUP/PSPP

Quiz navigation

1 ✓ 2 ✓ 3 ✓ 4 ✓ 5 ✓ 6 ✓
7 ✓ 8 ✓ 9 ✓ 10 ✓

Show one page at a time

Finish review

Status	Finished
Started	Monday, 28 September 2026, 10:45 AM
Completed	Monday, 28 September 2026, 11:27 AM
Duration	42 mins 9 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**
Correct
Mark 1.00 out of 1.00
Flag question

Consider the below words as key words and check the given input is key word or not.

keywords: {break, case, continue, default, defer, else, for, func, goto, if, map, range, return, struct, type, var}

Input format:

Take string as an input from stdin.

Output format:

Print the word is key word or not.

Example Input:

break

Output:

break is a keyword

Example Input:

IF

Output:

IF is not a keyword

For example:

Input	Result
`break`	`break is a keyword`
`IF`	`IF is not a keyword`

Answer: (penalty regime: 0 %)

```
keywords = {"break", "case", "continue", "default", "defer", "else",
            "for", "func", "goto", "if", "map", "range", "return",
            "struct", "type", "var"}

word = input()

if word in keywords:
    print(word, "is a keyword")
else:
    print(word, "is not a keyword")
```

	Input	Expected	Got	
✓	`break`	`break is a keyword`	`break is a keyword`	✓
✓	`IF`	`IF is not a keyword`	`IF is not a keyword`	✓

Passed all tests! ✓

Correct
Marks for this submission: 1.00/1.00.

Question **2**
Correct
Mark 1.00 out of 1.00
Flag question

The program must accept **N** series of keystrokes as string values as the input. The character **^** represents undo action to clear the last entered keystroke. The program must print the string typed after applying the undo operations as the output. If there are no characters in the string then print **-1** as the output.

Boundary Condition(s):

1 <= N <= 100
1 <= Length of each string <= 100

Input Format:

The first line contains the integer N.
The next N lines contain a string on each line.

Output Format:

The first N lines contain the string after applying the undo operations.

Example Input/Output 1:

Input:

3
Hey ^ goooo^^glee^
lucke^y ^charr^ms
ora^^nge^^^^

Output:

Hey google
luckycharms
-1

Answer: (penalty regime: 0 %)

```
n = int(input())

for i in range(n):
    s = input()
    a = []

    for x in s:
        if x == '^':
            if a:
                a.pop()
        else:
            a.append(x)

    print(''.join(a) if a else -1)
```

	Input	Expected	Got	

✓	`3`
	`Hey ^ goooo^^glee^`
	`lucke^y ^charr^ms`
	`ora^^nge^^^^`
		`Hey google`
		`luckycharms`
		`-1`
			`Hey google`
			`luckycharms`
			`-1`
				✓

Passed all tests! ✓

Correct
Marks for this submission: 1.00/1.00.

Question **3**
Correct
Mark 1.00 out of 1.00
Flag question

Find if a String2 is substring of String1. If it is, return the index of the first occurrence. else return -1.

Sample Input 1

thistest123string

123

Sample Output 1

8

Answer: (penalty regime: 0 %)

```
s1 = input()
s2 = input()

print(s1.find(s2))
```

	Input	Expected	Got	
✓	`thistest123string` `123`	`8`	`8`	✓

Passed all tests! ✓

Correct
Marks for this submission: 1.00/1.00.

Question **4**
Correct
Mark 1.00 out of 1.00
Flag question

Given a string S which is of the format USERNAME@DOMAIN.EXTENSION, the program must print the EXTENSION, DOMAIN, USERNAME in the reverse order.

Input Format:

The first line contains S.

Output Format:

The first line contains EXTENSION.
The second line contains DOMAIN.
The third line contains USERNAME.

Boundary Condition:

1 <= Length of S <= 100

Example Input/Output 1:

Input:

abcd@gmail.com

Output:

com
gmail
abcd

For example:

Input	Result

`arvijayakumar@rajalakshmi.edu.in`	`edu.in`
	`rajalakshmi`
	`arvijayakumar`

Answer: (penalty regime: 0 %)

```
s = input()

username, domain_ext = s.split('@')
domain, extension = domain_ext.split('.', 1)

print(extension)
print(domain)
print(username)
```

	Input	Expected	Got	

✓	`abcd@gmail.com`	`com`
		`gmail`
		`abcd`
			`com`
			`gmail`
			`abcd`
				✓

✓	`arvijayakumar@rajalakshmi.edu.in`	`edu.in`
		`rajalakshmi`
		`arvijayakumar`
			`edu.in`
			`rajalakshmi`
			`arvijayakumar`
				✓

Passed all tests! ✓

Correct
Marks for this submission: 1.00/1.00.

Question **5**
Correct
Mark 1.00 out of 1.00
Flag question

Write a Python program to get one string and reverses a string. The input string is given as an array of characters `char[]`.

You may assume all the characters consist of printable ascii characters.
Example 1:

```
Input:
hello
Output:
olleh
```

Example 2:

```
Input:
Hannah
Output:
hannaH
```

Answer: (penalty regime: 0 %)

```
s = input()
print(s[::-1])
```

	Input	Expected	Got	
✓	`hello`	`olleh`	`olleh`	✓
✓	`Hannah`	`hannaH`	`hannaH`	✓

Passed all tests! ✓

Correct
Marks for this submission: 1.00/1.00.

Question **6**
Correct
Mark 1.00 out of 1.00
Flag question

A pangram is a sentence where every letter of the English alphabet appears at least once.

Given a string sentence containing only lowercase English letters, return true if sentence is a pangram, or false otherwise.

Example 1:

Input:

thequickbrownfoxjumpsoverthelazydog

Output:

true

Explanation: sentence contains at least one of every letter of the English alphabet.

Example 2:

Input:

arvijayakumar

Output: false

Constraints:

1 <= sentence.length <= 1000

sentence consists of lowercase English letters.

For example:

Test	Result
`print(checkPangram('thequickbrownfoxjumpsoverthelazydog'))`	`true`
`print(checkPangram('arvijayakumar'))`	`false`

Answer: (penalty regime: 0 %)

Reset answer

```
def checkPangram(sentence: str) -> str:
    return str(len(set(sentence)) == 26).lower()
```

	Test	Expected	Got	
✓	`print(checkPangram('thequickbrownfoxjumpsoverthelazydog'))`	`true`	`true`	✓
✓	`print(checkPangram('arvijayakumar'))`	`false`	`false`	✓

Passed all tests! ✓

Correct
Marks for this submission: 1.00/1.00.

Question **7**
Correct
Mark 1.00 out of 1.00
Flag question

Assume that the given string has enough memory.

Don't use any extra space(IN-PLACE)

Sample Input 1

a2b4c6

Sample Output 1

aabbbbcccccc

Answer: (penalty regime: 0 %)

```
s = input()
result = ""

i = 0
while i < len(s):
    ch = s[i]
    i += 1

    num = ""
    while i < len(s) and s[i].isdigit():
        num += s[i]
        i += 1

    result += ch * int(num)

print(result)
```

	Input	Expected	Got	
✓	`a2b4c6`	`aabbbbcccccc`	`aabbbbcccccc`	✓
✓	`a12b3d4`	`aaaaaaaaaaaabbbdddd`	`aaaaaaaaaaaabbbdddd`	✓

Passed all tests! ✓

Correct
Marks for this submission: 1.00/1.00.

Question **8**
Correct
Mark 1.00 out of 1.00
Flag question

Given a string, determine if it is a palindrome, considering only alphanumeric characters and ignoring cases.

Note: For the purpose of this problem, we define empty string as valid palindrome.

Example 1:

```
Input:
A man, a plan, a canal: Panama

Output:
1
```

Example 2:

```
Input:
race a car

Output:
0
```

Constraints:

- `s` consists only of printable ASCII characters.

Answer: (penalty regime: 0 %)

```
s = input()

a = ""

for ch in s:
    if ch.isalnum():
        a += ch.lower()

if a == a[::-1]:
    print(1)
else:
    print(0)
```

	Input	Expected	Got	
✓	`A man, a plan, a canal: Panama`	`1`	`1`	✓
✓	`race a car`	`0`	`0`	✓

Passed all tests! ✓

Correct
Marks for this submission: 1.00/1.00.

Question **9**
Correct
Mark 1.00 out of 1.00
Flag question

Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

Open brackets must be closed by the same type of brackets.

Open brackets must be closed in the correct order.

Constraints:

1 <= s.length <= 10^4

s consists of parentheses only '()[]{}'.

For example:

Test	Result
`print(ValidParenthesis("()"))`	`true`
`print(ValidParenthesis("()[]{}"))`	`true`
`print(ValidParenthesis("(]"))`	`false`

Answer: (penalty regime: 0 %)

Reset answer

```
def ValidParenthesis(s: str) -> str:
    stack = []
    mapping = {")": "(", "}": "{", "]": "["}

    for char in s:
        if char in mapping:
            top_element = stack.pop() if stack else '#'
            if mapping[char] !=top_element:
                return "false"
        else:
            stack.append(char)

    return "true" if not stack else "false"
```

	Test	Expected	Got	
✓	`print(ValidParenthesis("()"))`	`true`	`true`	✓
✓	`print(ValidParenthesis("()[]{}"))`	`true`	`true`	✓
✓	`print(ValidParenthesis("(]"))`	`false`	`false`	✓

Passed all tests! ✓

Correct
Marks for this submission: 1.00/1.00.

Question **10**
Correct
Mark 1.00 out of 1.00
Flag question

Given a **non-empty** string s and an abbreviation abbr, return whether the string matches with the given abbreviation.

A string such as "word" contains only the following valid abbreviations:

["word", "1ord", "w1rd", "wo1d", "wor1", "2rd", "w2d", "wo2", "1o1d", "1or1", "w1r1", "1o2", "2r1", "3d", "w3", "4"]

Notice that only the above abbreviations are valid abbreviations of the string "word". Any other string is not a valid abbreviation of "word".

Note:

Assume s contains only lowercase letters and abbr contains only lowercase letters and digits.

Example 1:

Input

internationalization

i12iz4n

Output

true

Explanation

Given **s** = "internationalization", **abbr** = "i12iz4n":

Return true.

Example 2:

Input

apple

a2e

Output

false

Explanation

Given **s** = "apple", **abbr** = "a2e":

Return false.

Answer: (penalty regime: 0 %)

```
s = input()
a = input()

i = 0
j = 0

while j < len(a):
    if a[j].isdigit():
        n = 0
        while j < len(a) and a[j].isdigit():
            n = n * 10 + int(a[j])
            j += 1
        i += n
    else:
        if i >= len(s) or s[i] != a[j]:
            print("false")
            break
        i += 1
        j += 1
else:
    print("true" if i == len(s) else "false")
```

	Input	Expected	Got	
✓	`internationalization` `i12iz4n`	`true`	`true`	✓
✓	`apple` `a2e`	`false`	`false`	✓

Passed all tests! ✓

Correct
Marks for this submission: 1.00/1.00.

Finish review